ASSIGNMENT 2- UCS645

EXERCISE:

Question 1: Molecular Dynamics - Force Calculation

Problem: Implement parallel computation of Lennard-Jones potential forces in a molecular dynamics simulation. Given N particles in 3D space, calculate the total potential energy and forces acting on each particle.

Tasks:

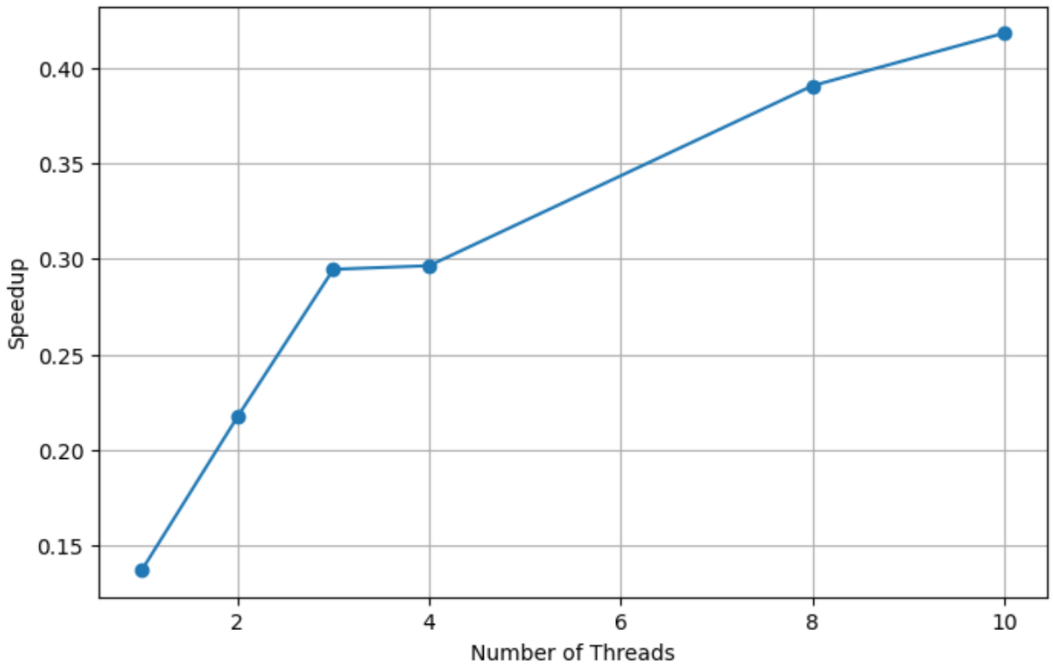
1. Parallelize the nested loops using OpenMP
2. Handle race conditions in force accumulation
3. Implement reduction for total energy
4. Optimize load balancing
5. Add performance measurement

OUTPUT:

```
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment2$ g++ -O3 -fopenmp -pg a2q1.cpp -o a2q1
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment2$ export OMP_NUM_THREADS=1; ./a2q1
MD Force Calculation Results:
Threads: 1
Speedup: 0.1373x
Efficiency: 13.7270%
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment2$ export OMP_NUM_THREADS=2; ./a2q1
MD Force Calculation Results:
Threads: 2
Speedup: 0.2175x
Efficiency: 10.8772%
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment2$ export OMP_NUM_THREADS=3; ./a2q1
MD Force Calculation Results:
Threads: 3
Speedup: 0.2945x
Efficiency: 9.8176%
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment2$ export OMP_NUM_THREADS=4; ./a2q1
MD Force Calculation Results:
Threads: 4
Speedup: 0.2964x
Efficiency: 7.4104%
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment2$ export OMP_NUM_THREADS=8; ./a2q1
MD Force Calculation Results:
Threads: 8
Speedup: 0.3904x
Efficiency: 4.8806%
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment2$ export OMP_NUM_THREADS=10; ./a2q1
MD Force Calculation Results:
Threads: 10
Speedup: 0.4179x
Efficiency: 4.1792%
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment2$
```

```
Performance counter stats for './a2q1':

          2913.80 msec task-clock:u                     #    6.906 CPUs utilized
                0      context-switches:u               #    0.000 /sec
                0      cpu-migrations:u                 #    0.000 /sec
              279      page-faults:u                    #   95.751 /sec

      0.421930958 seconds time elapsed

      2.890203000 seconds user
      0.015902000 seconds sys
```

Threads	Speedup	Efficiency (%)
1	0.1373	13.7270
2	0.2175	10.8772
3	0.2945	9.8176
4	0.2964	7.4104
8	0.3904	4.8806
10	0.4179	4.1792



The speedup increases gradually as the number of threads increases, starting from **0.1373× at 1 thread** and reaching a maximum of **0.4179× at 10 threads**. This shows that parallel execution does improve performance, but the improvement is **not close to ideal scaling**. The efficiency decreases continuously from **13.7270% (1 thread)** to **4.1792% (10 threads)**, indicating that as more threads are added, the overhead of parallelization (such as thread creation, synchronization, and memory access contention) dominates the benefits.

Question 2: Bioinformatics - DNA Sequence Alignment (Smith-Waterman)

Problem: Implement a parallel version of the Smith-Waterman local sequence alignment algorithm for comparing two DNA sequences.

Tasks:

1. Parallelize the scoring matrix computation
2. Handle anti-dependencies in the dynamic programming approach
3. Experiment with different scheduling strategies
4. Implement wavefront parallelization (advanced)

OUTPUT:

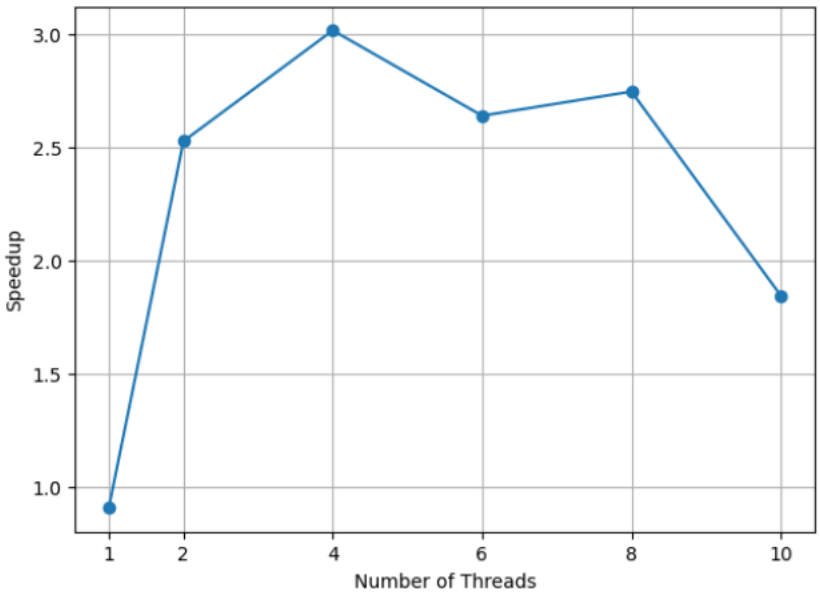
```
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment2$ g++ -O3 -fopenmp -pg a2q2.cpp -o a2q2
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment2$ export OMP_NUM_THREADS=1; ./a2q2
Sequence Alignment Results:
Threads: 1

Scheduling Strategy Performance:
  static: 0.1273s
  dynamic: 0.5028s
  guided: 0.1255s

Best Performance:
Speedup: 0.9097x
Efficiency: 90.9743%
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment2$ export OMP_NUM_THREADS=2; ./a2q2
Sequence Alignment Results:
Threads: 2

Scheduling Strategy Performance:
  static: 0.0445s
  dynamic: 1.1114s
  guided: 0.0738s

Best Performance:
Speedup: 2.5257x
Efficiency: 126.2875%
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment2$ export OMP_NUM_THREADS=4; ./a2q2
Sequence Alignment Results:
Threads: 4

Scheduling Strategy Performance:
  static: 0.0374s
  dynamic: 0.8400s
  guided: 0.0840s

Best Performance:
Speedup: 3.0141x
Efficiency: 75.3527%
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment2$ export OMP_NUM_THREADS=6; ./a2q2
Sequence Alignment Results:
Threads: 6

Scheduling Strategy Performance:
  static: 0.0434s
  dynamic: 0.7959s
  guided: 0.0850s

Best Performance:
Speedup: 2.6380x
Efficiency: 43.9673%
```

```
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment2$ export OMP_NUM_THREADS=8; ./a2q2
Sequence Alignment Results:
Threads: 8

Scheduling Strategy Performance:
  static: 0.0558s
  dynamic: 1.1110s
  guided: 0.1488s

Best Performance:
Speedup: 2.7446x
Efficiency: 34.3079%
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment2$ export OMP_NUM_THREADS=10; ./a2q2
Sequence Alignment Results:
Threads: 10

Scheduling Strategy Performance:
  static: 0.0845s
  dynamic: 1.3151s
  guided: 0.1862s

Best Performance:
Speedup: 1.8453x
Efficiency: 18.4526%
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment2$
```

```
Performance counter stats for './a2q2':

         17416.97 msec task-clock:u                     #    8.758 CPUs utilized
                0      context-switches:u               #    0.000 /sec
                0      cpu-migrations:u                 #    0.000 /sec
            24645      page-faults:u                    #    1.415 K/sec

      1.988776490 seconds time elapsed

     17.299421000 seconds user
      0.112125000 seconds sys
```

Threads	Static Time (s)	Dynamic Time (s)	Guided Time (s)	Best Speedup	Efficiency (%)
1	0.1273	0.5028	0.1255	0.9097	90.9743
2	0.0445	1.1114	0.0738	2.5257	126.2875
4	0.0374	0.8400	0.0840	3.0141	75.3527
6	0.0434	0.7959	0.0850	2.6380	43.9673
8	0.0558	1.1110	0.1488	2.7446	34.3079
10	0.0845	1.3151	0.1862	1.8453	18.4526

The results show that performance improves significantly when increasing threads from 1 to 4, achieving the maximum speedup of **3.01× at 4 threads**, after which scaling becomes inconsistent. Beyond 4 threads, speedup fluctuates and drops at 10 threads due to increasing parallel overhead, memory contention, and synchronization costs. **Static scheduling consistently gives the best execution time**, indicating the workload is fairly uniform, while **dynamic scheduling performs worst** due to high scheduling overhead. Overall, the program achieves best parallel efficiency at lower thread counts, and the optimal configuration for this system is around **4 threads with static scheduling**.



Question 3: Scientific Computing - Heat Diffusion Simulation

Problem: Simulate heat diffusion in a 2D metal plate using finite difference method with parallel OpenMP implementation.

Hints for Question 3:

1. **Data Dependency Analysis**: The heat equation update has dependencies only on previous time step, allowing parallelization of spatial loops
2. **Reduction Operations**: Use reduction(+:total) for accumulating total heat
3. **Memory Optimization**: Consider loop tiling/cache blocking for better cache utilization
4. **Race Conditions**: No race conditions exist as each grid point writes to unique location in next_temperature.
5. **Scheduling Strategies**: Experiment with schedule(static), schedule(dynamic, chunk_size), and schedule(guided).

Profiling Tools (Exposure Level)

Use the following tools for observation and analysis:

- perf stat – to measure cycles, instructions, and cache misses
- LIKWID – to observe FLOPS and memory bandwidth

Analysis experiment tables and graphs and give discussion.

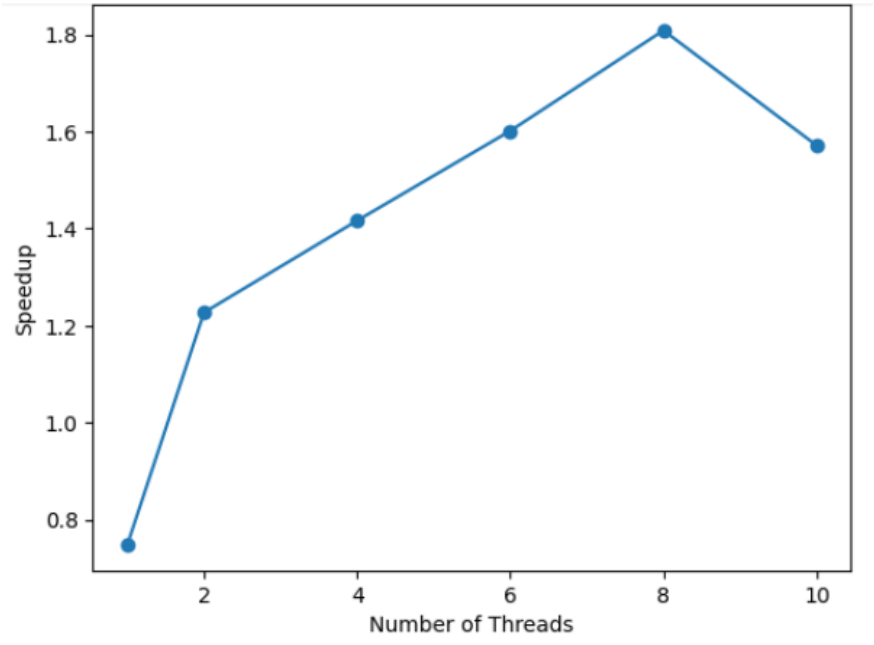
OUTPUT:

```
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment2$ g++ -O3 -fopenmp -pg a2q3.cpp -o a2q3
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment2$ export OMP_NUM_THREADS=1; ./a2q3
Heat Equation Solver Results:
Threads: 1

Scheduling Strategy Performance:
  static: 0.9646s
  dynamic-16: 0.9875s
  guided: 0.9578s

Best Performance:
Speedup: 0.7491x
Efficiency: 74.9115%
Bandwidth: 14.0135 GB/s
Total Heat (final): 83890668.3490
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment2$ export OMP_NUM_THREADS=2; ./a2q3
Heat Equation Solver Results:
Threads: 2

Scheduling Strategy Performance:
  static: 0.6330s
  dynamic-16: 0.8090s
  guided: 0.6457s

Best Performance:
Speedup: 1.2272x
Efficiency: 61.3591%
Bandwidth: 21.2020 GB/s
Total Heat (final): 83890668.3490
```

```
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment2$ export OMP_NUM_THREADS=4; ./a2q3
Heat Equation Solver Results:
Threads: 4

Scheduling Strategy Performance:
  static: 0.5573s
  dynamic-16: 0.5312s
  guided: 0.5357s

Best Performance:
Speedup: 1.4165x
Efficiency: 35.4118%
Bandwidth: 25.2685 GB/s
Total Heat (final): 83890668.3490
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment2$ export OMP_NUM_THREADS=6; ./a2q3
Heat Equation Solver Results:
Threads: 6

Scheduling Strategy Performance:
  static: 0.5746s
  dynamic-16: 0.4514s
  guided: 0.4655s

Best Performance:
Speedup: 1.6015x
Efficiency: 26.6920%
Bandwidth: 29.7314 GB/s
Total Heat (final): 83890668.3490
```

```
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment2$ export OMP_NUM_THREADS=8; ./a2q3
Heat Equation Solver Results:
Threads: 8

Scheduling Strategy Performance:
  static: 0.5641s
  dynamic-16: 0.4247s
  guided: 0.4325s

Best Performance:
Speedup: 1.8083x
Efficiency: 22.6035%
Bandwidth: 31.6045 GB/s
Total Heat (final): 83890668.3490
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment2$ export OMP_NUM_THREADS=10; ./a2q3
Heat Equation Solver Results:
Threads: 10

Scheduling Strategy Performance:
  static: 0.4673s
  dynamic-16: 0.4845s
  guided: 0.4614s

Best Performance:
Speedup: 1.5714x
Efficiency: 15.7140%
Bandwidth: 29.0890 GB/s
Total Heat (final): 83890668.3490
```

```
Performance counter stats for './a2q3':

         22718.52 msec task-clock:u                     #    7.141 CPUs utilized
                0      context-switches:u               #    0.000 /sec
                0      cpu-migrations:u                 #    0.000 /sec
            16550      page-faults:u                    #  728.481 /sec

      3.181481656 seconds time elapsed

     22.582033000 seconds user
      0.104527000 seconds sys
```



Threads	Static (s)	Dynamic-16 (s)	Guided (s)	Best Speedup	Efficiency (%)	Bandwidth (GB/s)
1	0.9461	0.9875	0.9578	0.7491	74.9115	14.0135
2	0.6330	0.8090	0.6457	1.2272	61.3591	21.2020
4	0.5573	0.5312	0.5357	1.4165	35.4118	25.2685
6	0.5746	0.4514	0.4655	1.6015	26.6920	29.7314
8	0.5641	0.4247	0.4325	1.8083	22.6035	31.6045
10	0.4673	0.4845	0.4614	1.5714	15.7140	29.0890

Speedup increases as thread count rises up to **8 threads**, showing effective parallelization and improved memory bandwidth utilization; however, **efficiency steadily drops**, indicating growing overheads and resource contention. After 8 threads, performance declines (speedup falls at 10 threads), suggesting the program becomes **memory-bound and suffers from parallel overhead**, meaning adding more threads beyond the optimal point reduces scalability benefits.

Vaani Prashar
3C16
102303159